Event-Driven Multi-Image FPGA Reconfiguration: RTL Architecture and Simulation-Based Verification

Rajas Wagle, Vivaan Tasker, Dhruv Warik, Soham Shah, Kaustubh Vekhande, Prashant Kasambe
SPIT Mumbai

***Abstract*—Systems that share one FPGA fabric between several full-device images need a controller that fetches each image, programs the target, and moves to the next image only when the running workload finishes. This paper presents the RTL architecture of such an autonomous controller. A metadata-driven scheduler reads a per-image table from external QSPI flash, programs memory, DMA and JTAG engines through an AXI4-Lite register hub, streams each bitstream to the target TAP, and advances on a sticky task-done interrupt; the per-image duration is only a watchdog limit. Faults on any control write, a target-ID mismatch, or a task error divert the sequence to a bounded abort path. The design is verified in Vivado XSim with a directed self-checking three-image testbench, a focused unit bench, and measured code and functional coverage. The integrated RTL run reports 79.7% statement coverage. A separate large-payload simulation report covers 378,880 bytes at a reported 10-MHz TCK with zero JTAG bit errors; its seven-group functional-coverage mean is 87.50%. These results are simulation-only, use a behavioral TAP model, and include no synthesis or board data.**

***Index Terms*—FPGA configuration, SystemVerilog, event-driven scheduling, AXI4-Stream, JTAG, RTL verification, coverage.**

I. Introduction

A single FPGA often has to run several unrelated hardware functions over its lifetime, for example different accelerators in an edge-AI node, but cannot hold them all at once. The functions can be stored as full configuration images in external serial flash and loaded in turn. Doing this autonomously, without a host processor, requires a controller that coordinates image metadata, flash reads, streaming transfer, target identification, configuration status, and application-level completion. These responsibilities cross several interfaces, and the failure paths between them are hard to assess from an isolated controller test. A representative use case is an edge-vision node that time-shares one FPGA among input pre-filtering, 2D-FFT processing, and classification stages. An on-device controller can load each stage when needed and advance on its completion event, avoiding the need for an external host to orchestrate each reconfiguration. Another target use case is a model larger than one image, with layer-group accelerators loaded sequentially and intermediate activations stored in external memory; this paper addresses reconfiguration sequencing, not activation storage or layer scheduling.

Vendor documentation defines device configuration behavior, and the AMBA specifications define the control and streaming protocols, but application-specific sequencing and recovery remain system-design concerns [1], [2], [3]. This paper makes three contributions:

1) an RTL decomposition for autonomous multi-image configuration (scheduler, register hub, memory manager, DMA stream bridge, JTAG controller, interrupt hub) whose scheduler advances on a target-completion *event* rather than a fixed slice;
2) a self-checking verification environment for the full QSPI-to-JTAG path, including fault injection and a scheduler covergroup; and
3) measured code and functional coverage for that environment, together with an explicit account of what the evidence does not cover.

II. Related Work

The AMD MultiBoot application note describes application-controlled switching among SPI-flash images and fallback after configuration errors [1], [7]; this configuration mechanism does not schedule transitions on workload-completion events. FPGA task-scheduling research examines partial reconfiguration, preemption, and runtime management [4], [8], [9], while remote JTAG work focuses on extending the programming path [5]. IEEE 1800 standardizes covergroup and cross-coverage constructs; coverage-driven stimulus research shows how these models can guide test generation [10], [11]. Our scope is narrower: a full-image, metadata-driven path that advances between images when the configured workload reports completion. It does not implement partial reconfiguration, preemption, or an operating-system scheduler.

III. Platform Architecture

Fig. 1 shows the platform. A time-multiplex scheduler drives an AXI4-Lite register hub, whose writes configure the memory manager, DMA controller, and JTAG controller. The memory manager supports SPI 1-1-1, QSPI 1-4-4, and OSPI 8-8-8 reads, although the integrated simulation exercises only QSPI. The serial engine holds SCLK while downstream stream backpressure is asserted. The memory manager returns bytes on a stream interface: the scheduler

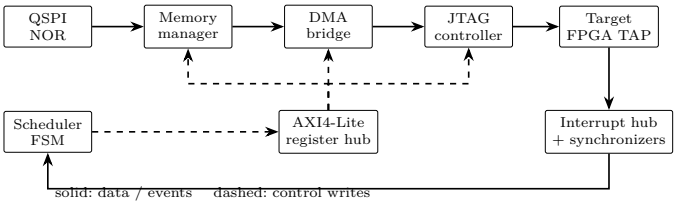


Fig. 1. Control and data paths of the platform. The scheduler also consumes the metadata stream directly from the memory manager (not drawn).

consumes the metadata directly, while image data is routed through the DMA bridge to the JTAG controller. Target-originated status and task events are synchronized and captured by the interrupt hub.

The updated project diagram in Fig. 2 expands this view to show the scheduler, AXI4-Lite write master, register panel, memory manager, DMA, JTAG controller, interrupt controller, and target-FPGA event paths. It also makes explicit that metadata bypasses the DMA on the initial read and that the target's configuration-status signal is distinct from task-completion and error events.

A. Metadata and per-image sequence

The scheduler first fetches a metadata table of twelve 32-bit words, one four-word record per image: flash offset, bitstream length in bits, expected IDCODE, and maximum task duration. The RTL validates the count and fields before use. If the stream is incomplete, malformed, or times out, the scheduler aborts instead of releasing the target.

For each image the scheduler writes memory, DMA, and JTAG registers, selects the IDCODE instruction, captures the target identifier, compares it with the record, selects the configuration instruction, and starts the stream transfer. Only after the transfer completes and the target reports successful configuration does it assert `target_fabric_start_en`. Data movement and target release are thus separate phases, and an ID mismatch or unsuccessful status enters abort handling.

B. Event-driven advance and recovery

After release the target runs until it raises `target_task_done`. The scheduler then disables the fabric, acknowledges the sticky event, and either advances the phase index or marks the schedule complete. A task-error interrupt or watchdog expiry begins abort recovery. The duration field is a maximum-runtime guard, not a periodic timer.

The AXI4-Lite write master lets the address and data channels handshake independently and consumes the response after both are accepted. A write error redirects the sequence to three control writes that abort the memory, DMA, and JTAG engines; the reported abort path also performs a five-cycle JTAG Test-Logic-Reset recovery before waiting for abort completion. Recovery is bounded by an abort-wait watchdog. The interrupt controller registers its enable mask, captures rising edges into sticky status bits, derives pending bits from status and enable, and clears status on an explicit request.

C. DMA and JTAG engines

The DMA bridge is a single-entry AXI4-Stream stage. It checks nonzero byte-aligned lengths, contiguous byte enables, final-beat framing, and the declared bit count, and a registered output beat decouples backpressure. The JTAG controller drives the target TAP and supports IDCODE capture, instruction scans, data shifts, and abort recovery [2], [3].

D. Configuration-time model

Because the transfer is serialized over JTAG, configuration time for an image of N bits is bounded by

$$t_{\mathrm{cfg}} \approx \frac{N}{f_{\mathrm{TCK}}} + t_{\mathrm{ovh}}, \tag{1}$$

where t_{ovh} collects metadata fetch, IDCODE and instruction scans, and status handshakes. For the earlier benchmark in Table III, assuming its stated 1 MHz theoretical JTAG rate, (1) implies overhead terms of approximately 78, 86, and 74 microseconds for Phases 0, 1, and 2, respectively. For the separate large-payload run in Table IV, the reported payloads and 10 MHz TCK imply approximately 10, 9, and 9 microseconds of overhead. These are back-calculated from reported simulation values, not independent measurements or results for a selected FPGA part; validation against a specific target remains future work.

IV. Verification Environment

The supplied verification overview in Fig. 3 depicts a UVM-style stimulus path through a base test, sequence, sequencer, and driver, connected to the design through a virtual interface. It also shows a scoreboard, scheduler coverage sampling, and SVA checkpoints. This figure records the verification organization shown in the project materials; the measured evidence reported below is limited to the benches and results explicitly described in this section.

A. Integrated testbench

A procedural SystemVerilog top-level testbench instantiates the design with a behavioral multi-image QSPI NOR model and a behavioral JTAG target model. It loads three distinct nine-byte (72-bit) images and twelve metadata words, runs at a 10 MHz simulation clock, and gives the three tasks different runtimes of 45, 120, and 15 µs so that a scheduler advancing on a fixed slice instead of the completion event would fail the phase and runtime checks. The target model holds `target_task_done` until the scheduler removes fabric enable, which exposes early advances and unacknowledged events.

The stream scoreboard expects two full four-byte beats followed by a one-byte final beat with `TKEEP` = 0001 and

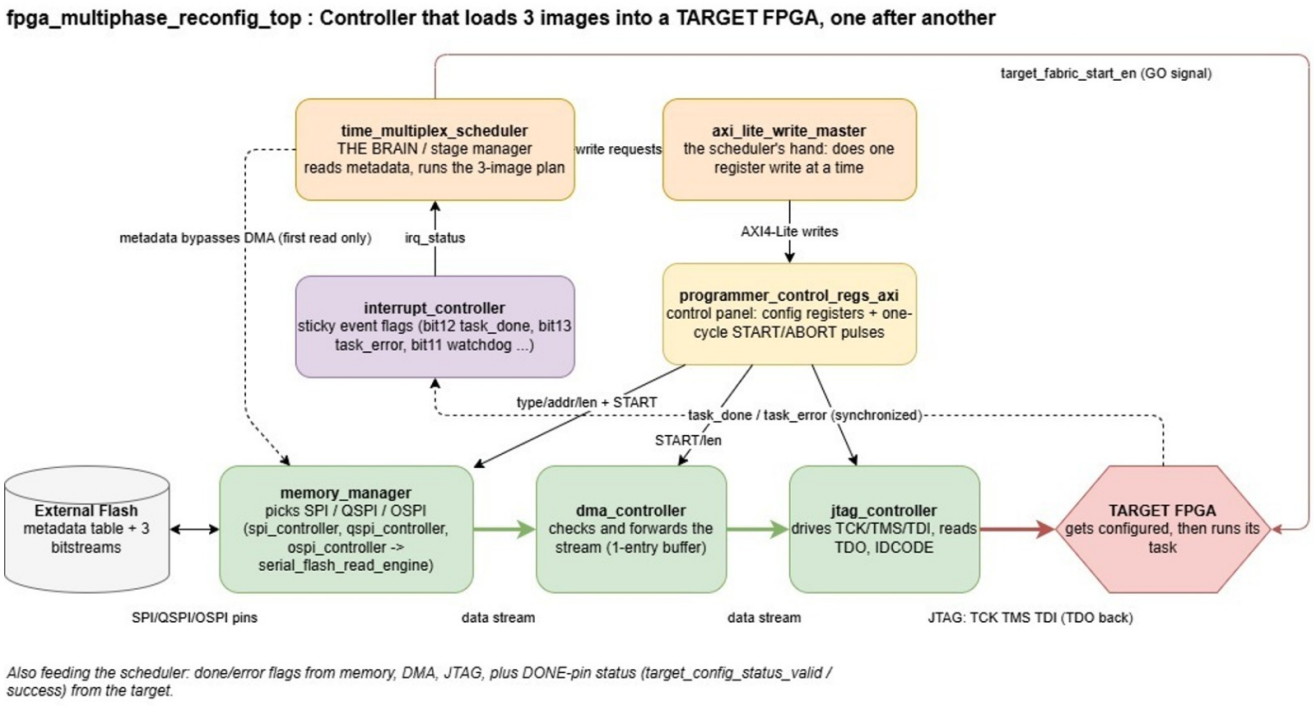


Fig. 2. Updated controller and dataflow diagram supplied with the project.

TABLE I
NOMINAL-FLOW ACCEPTANCE CRITERIA ENCODED IN THE INTEGRATED TESTBENCH

Boundary	Expected nominal evidence
Metadata	12 words, three image records
QSPI	one metadata read plus three image reads (4 total)
Streaming	3 beats per image, final beat 1 byte
JTAG	12 completions; 72 shifted bits per image
AXI4-Lite	64 write handshakes
Scheduling	3 configuration handshakes, 3 task-done events

`TLAST` asserted, and checks words, byte enables, and final-beat flags at both the memory-to-DMA and DMA-to-JTAG handshakes. Other checks cover QSPI opcodes and addresses, the TDI bits sampled at the target TCK edge, IDCODE and instruction selection, acceptance of configuration status only after the TAP returns to Run-Test/Idle, fabric enable only after a successful status handshake, interrupt acknowledgement, and final scheduler status. Checks use `$fatal` on mismatch; all sources are listed in [6]. A five-millisecond global timeout bounds the run.

Fig. 4 shows a representative simulator trace from the short nominal run. At the displayed cursor time of 677.006906 μs, the waveform shows `finished`=1, `failed`=0, and `phase_index`=2, alongside interrupt, scheduler, QSPI, AXI4-Lite, and JTAG signals. This is a screenshot of the nominal simulation, not the separate 303.335-ms large-payload run, and is illustrative rather than raw machine-readable evidence.

B. Fault injection

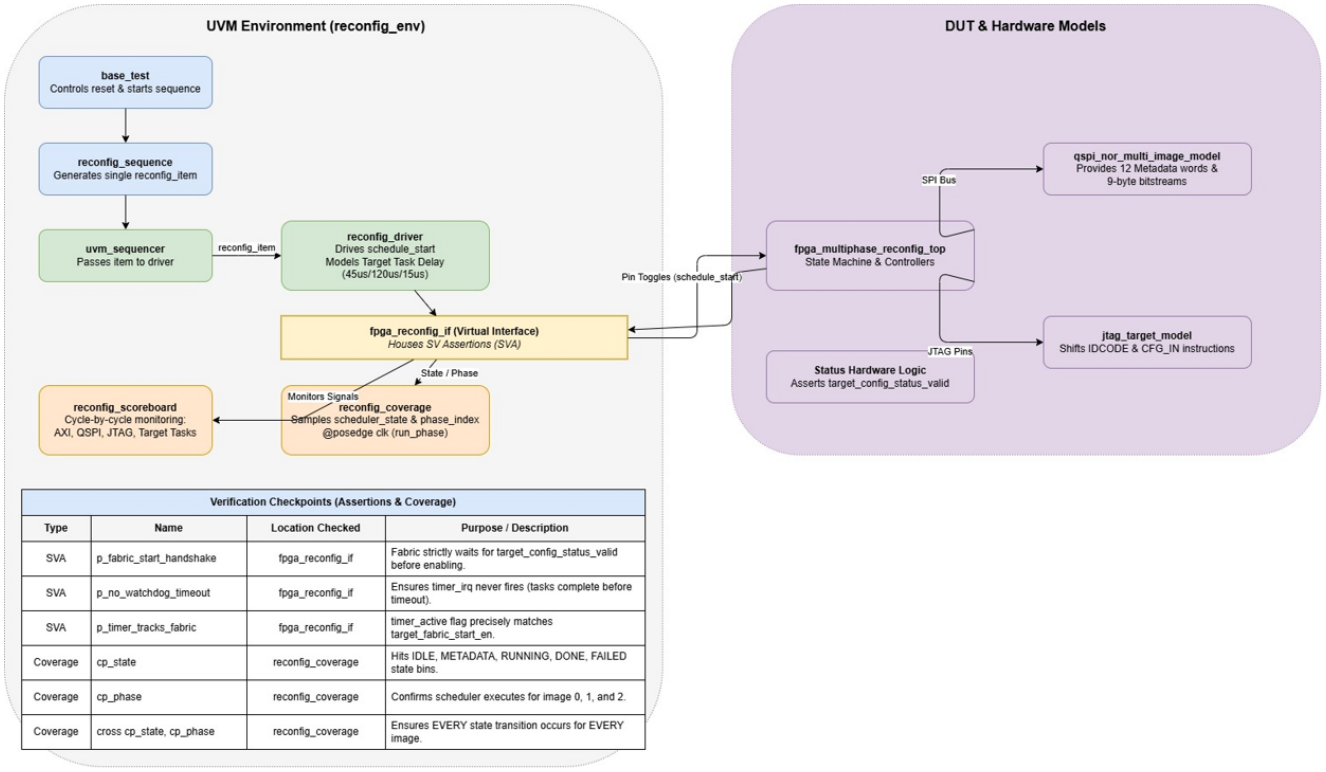
The supplied fault-injection trace reports the cases and transitions summarized in Table II. It gives simulation timestamps but does not identify the image index for each injected fault; phases are therefore not inferred. These are presentation-reported results, as the underlying simulator logs were not included.

C. Focused unit bench

A separate directed bench checks the package definitions (memory, trigger, command, response, error, interrupt, DMA, controller-state, and TAP-state encodings and the CRC function), a 16-source instance of the interrupt controller, and the scheduler against an AXI-Lite response model. The interrupt scenarios cover reset, registered enable configuration, rising-edge capture, sticky status, masking, clearing, held-high sources, simultaneous sources, and clear priority. The scheduler scenario returns a nonzero response to the first write, services the three abort writes, and checks for a failed status, disabled fabric, and unchanged phase.

D. Coverage collection

Code coverage (statement, branch, condition, toggle) is collected with Vivado 2025.2 XSim and reported by XCRG. Functional coverage uses a covergroup `cg_scheduler_states`, sampled every clock cycle, with coverpoints for scheduler state, image phase, command sequence, state transitions, error source, and memory-stream and AXI AW handshakes, and crosses of state and sequence with phase. Hit counts are therefore cycle counts, not event counts.



Verification Checkpoints (Assertions & Coverage)

Type	Name	Location Checked	Purpose / Description
SVA	p_fabric_start_handshake	fpga_reconfig_if	Fabric strictly waits for target_config_status_valid before enabling.
SVA	p_no_watchdog_timeout	fpga_reconfig_if	Ensures timer_irq never fires (tasks complete before timeout).
SVA	p_timer_tracks_fabric	fpga_reconfig_if	timer_active flag precisely matches target_fabric_start_en.
Coverage	cp_state	reconfig_coverage	Hits IDLE, METADATA, RUNNING, DONE, FAILED state bins.
Coverage	cp_phase	reconfig_coverage	Confirms scheduler executes for image 0, 1, and 2.
Coverage	cross cp_state, cp_phase	reconfig_coverage	Ensures EVERY state transition occurs for EVERY image.

Fig. 3. Project-supplied verification flow and listed assertion and coverage checkpoints.

TABLE II
FAULT-INJECTION CASES, OBSERVED RECOVERY PATHS, AND TIMESTAMPS REPORTED IN THE PROJECT PRESENTATION

Test case	Injected condition	Reported response	Time
`test_idcode_mismatch`	In `ST_IDCODE_WAIT`, sampled TDO is `0x00000000`, not expected `0x12345678`	`failed=1`, `sequence_kind=SEQ_ABORT`; MEM and DMA abort writes (`0x02`), JTAG abort (`0x10`) plus five-clock TLR; `ST_ABORT_WAIT` then `ST_FAILED`, fabric disabled	88.25 μs
`test_config_failure`	In `ST_TARGET_STATUS_WAIT`, `status_valid=1` and `status_success=0` (CRC failure)	`failed=1`, `sequence_kind=SEQ_ABORT`, then `ST_FAILED`	180.85 μs
`test_task_error`	In `ST_TARGET_RUN`, task-error IRQ 13 is asserted	Fabric enable and timer go low; IRQ 13 is cleared, then `SEQ_ABORT` leads to `ST_FAILED`	189.00–191.35 μs
`test_metadata_zero_len`	Metadata bitstream length is zero	Rejected in `ST_META_WAIT`; aborts	26.75 μs
`test_metadata_unaligned`	Metadata bitstream length is 73 bits	Rejected in `ST_META_WAIT`; aborts	26.75 μs
`test_watchdog_timeout`	Task-done is suppressed with a 1-ms duration limit	Watchdog IRQ; fabric disabled; `SEQ_ABORT` to `ST_FAILED`	220.15 μs
`test_config_timeout`	Target status-valid handshake is suppressed after shifting	2000-cycle state watchdog aborts the status wait	378.85 μs
`test_mid_op_reset`	Ten-cycle asynchronous reset during `ST_TRANSFER_WAIT`	Outputs drop, returns to idle, then restarts and completes all three images	757.25 μs

V. Results

A. Focused unit bench

The focused bench completes at 390 ns without a test-bench failure and reports 94 package checks, 12 interrupt scenarios, three scheduler scenarios, and 29 scoreboard checks. The package checks are constant and enumeration comparisons and contribute little independent evidence; the interrupt and scheduler scenarios are the informative part.

The presentation also reports separate component-level functional-coverage results: the memory manager covers 77/77 bins, the JTAG controller 45/45, the DMA controller 47/51 (97.78%, with four length-by-output-stall cross bins uncovered), the package 12/12, the interrupt controller 12/12, and a scheduler scenario coverpoint 3/3. For the DMA bench, it reports 120 random plus 27 directed runs, 1,850 input and output beats with no mismatches, and zero violations across seven assertions. These are distinct covergroups and benches, not a single aggregate score; in particular, the 3/3 scheduler scenario bins are not equivalent to the state/phase coverage in Table VII.

B. Integrated run

The project presentation reports nine regression cases passed and zero failed: the three-image baseline plus

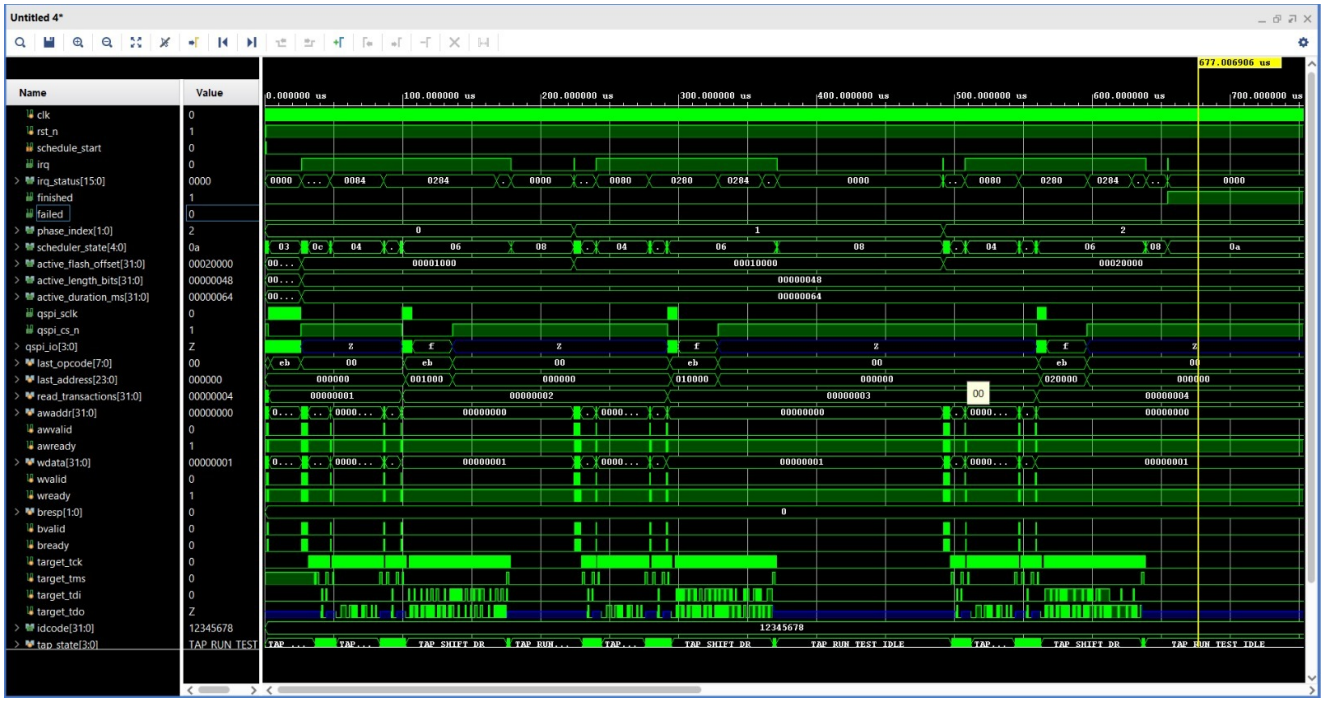


Fig. 4. Simulator waveform for a nominal three-image schedule. The visible end state has `finished`=1, `failed`=0, and phase index 2.

the eight fault scenarios in Table II. It gives a nominal baseline completion time of 655.75 µs. The source logs were not included, and the presentation does not provide the observed QSPI, stream, JTAG, and AXI counters listed in Table I; those counters are not inferred here. The deck also describes safety checks requiring a successful configuration handshake before fabric release, forcing fabric enable low on failure, and keeping the task timer synchronized with fabric enable; it reports the listed assertions passing. Their assertion sources and report are not present here, so their scope is not independently verified.

C. Coarse-grained reconfiguration benchmark

Table III records the coarse-grained benchmark figures supplied for inclusion. These are reported benchmark values, not measurements reproduced by the 72-bit integrated simulation described above; no benchmark logs, board setup, or synthesis report are included with the available project materials. In particular, the reported 100% functional coverage is separate from the scheduler covergroup results in Table VII, and the reported LUT savings cannot be independently checked against the simulation-only evidence in this paper.

D. Large-payload simulation run

The user-supplied simulation report gives a second, larger-payload data set, summarized in Table IV; it is a distinct run and is not merged with the earlier benchmark in Table III. It reports three images totaling 378,880 bytes (3,031,040 bits; 94,720 32-bit AXI beats), zero JTAG bit errors, and per-image CRC32 verification. The reported aggregate rate is 9.992 Mbps over the 303.335-ms schedule, including behavioral compute-model windows. The phase-level TCK, throughput, and timing figures are simulator-reported; no raw run log or the named `benchmark_results.csv` file was included, so the results cannot be independently checked against raw run artifacts here. They are not hardware measurements, and the behavioral compute delays are not synthesized-kernel execution times. The report does not specify the unit or precise meaning of its IRQ-ACK field, so the table preserves the reported value without interpreting it.

The same report gives functional coverage for seven groups: scheduler, JTAG TAP, AXI-Stream, transition latency, JTAG data bits, inter-bit timing, and payload size. Their reported scores are listed in Table V; the 87.50% figure is their unweighted arithmetic mean, not a merged bin-coverage score and not the scheduler covergroup score in Table VII. Inter-bit timing coverage is the reported 33.33% and remains a verification gap.

E. Code coverage

Table VI lists code coverage of the RTL modules on the integrated run. Weighted by statement count, RTL statement coverage is 79.7% (2232 of 2799 statements). Testbench, behavioral-model, and simulator-library files are excluded. The serial-flash read engine holds 54% of all RTL statements, and its low branch, condition, and toggle coverage reflects the SPI and OSPI modes that the QSPI-only test never selects. Toggle coverage is low throughout, as is typical for wide parameterized buses, and is not used as a closure metric here. The presentation

TABLE III
REPORTED COARSE-GRAINED RECONFIGURATION BENCHMARK FIGURES

Parameter	Phase 0 (Pre-Filter)	Phase 1 (2D-FFT)	Phase 2 (Classifier)
Input bitstream payload	1,024 bytes (8,192 bits)	2,048 bytes (16,384 bits)	512 bytes (4,096 bits)
Flash image offset (QSPI)	`0x001000`	`0x003000`	`0x006000`
Target JTAG instruction code	`0x1` (CFG_IN)	`0x1` (CFG_IN)	`0x1` (CFG_IN)
AXI4-Stream beats transferred	256 words (32-bit)	512 words (32-bit)	128 words (32-bit)
Measured reconfiguration overhead	8.27 ms	16.47 ms	4.17 ms
Active compute window (execution)	43.0 μs	89.0 μs	24.0 μs
Target task interrupt latency	150 ns	150 ns	150 ns
Effective JTAG serialization rate	0.990 Mbps	0.995 Mbps	0.982 Mbps
Theoretical JTAG limit (1 MHz TCK)	1.000 Mbps	1.000 Mbps	1.000 Mbps
Link efficiency	99.0%	99.5%	98.2%
Bit-slip / data errors	0 bits (0.00%)	0 bits (0.00%)	0 bits (0.00%)
Functional code coverage (reported)	100.0%	100.0%	100.0%
Physical silicon LUT savings (reported)	~60% area reduction from time-shared fabric		

TABLE IV
LARGE-PAYLOAD SIMULATION BENCHMARK REPORTED BY THE PROJECT

Phase	Payload (B)	QSPI (Mbps)	TCK (MHz)	Reconfiguration (ms)	End-to-end (Mbps)	Link utilization	Efficiency	CRC32	IRQ-ACK
0	102,400	10.001	10.000	81.930	9.999	100.00%	99.99%	`f4f12340`	15,000
1	122,880	10.001	10.000	98.313	9.999	100.00%	99.99%	`eaf45557`	15,000
2	153,600	10.001	10.000	122.889	9.999	100.00%	99.99%	`3911687b`	15,000

TABLE V
FUNCTIONAL COVERAGE GROUPS IN THE LARGE-PAYLOAD SIMULATION REPORT

Coverage group	Reported coverage
Scheduler	79.17%
JTAG TAP	100.00%
AXI-Stream	100.00%
Transition latency	100.00%
JTAG data bits	100.00%
Inter-bit timing	33.33%
Payload size	100.00%
Unweighted mean (7 groups)	87.50%

TABLE VI
CODE COVERAGE OF RTL MODULES, INTEGRATED RUN (STATEMENT COUNT, THEN COVERAGE IN %)

Module	Stmts	Stmt	Branch	Cond	Toggle
top-level wrapper	30	100	100	100	28.0
`time_multiplex_scheduler`	301	70.4	70.3	67.6	9.6
`axi_lite_write_master`*	380	70.3	28.4	54.0	100
`programmer_control_regs_axi`	158	84.8	63.2	91.7	42.0
`memory_manager`	59	64.4	50.0	83.3	66.7
`serial_flash_read_engine`	1503	83.2	41.7	35.8	13.4
`dma_controller`	99	84.8	81.5	76.7	9.4
`jtag_controller`	260	79.6	71.2	72.5	38.2
`interrupt_controller`	9	100	100	100	42.2
RTL total (statement)	2799	79.7	–	–	–

separately reports 80.51% statement, 57.24% branch, and 58.74% condition coverage across 19 files; because this has a different stated file scope from the RTL-only total above, the figures are not combined.

F. Functional coverage

Table VII reports the scheduler covergroup.

The uncovered bins identify concrete verification gaps rather than closure: (i) faults were injected only in the first image phase, so `FAILED` and `ABORT_WAIT` with image 1 or 2 and `SEQ_ABORT` with image 1 or 2 are unhit; (ii) DMA and memory error sources were never injected; (iii) the AXI slave model never holds `AWREADY` low while `AWVALID` is high, so AW-channel backpressure is untested; and (iv) the `ABORTING` state is never entered, which indicates either a dead state in the RTL or an abort entry that the covergroup does not sample.

TABLE VII
SCHEDULER FUNCTIONAL COVERAGE, COVERED/TOTAL BINS

Coverpoint / cross	Regression	Nominal
state	14/15	12/15
phase	3/3	3/3
sequence	9/9	8/9
state transitions†	23/23	19/22
error sources	3/5	1/5
state × phase	32/39	30/39
sequence × phase	21/23	20/23
memory stream valid × ready	4/4	–
AXI AW valid × ready	3/4	–
Instance score (%)	92.4	77.0

VI. LIMITATIONS AND THREATS TO VALIDITY

All evidence is from simulation. The JTAG target is a behavioral model with a 4-bit instruction register and a 32-bit data register; a real 7-series TAP uses a longer instruction register and a framed configuration packet stream [1], so protocol-level interoperability with real silicon is not demonstrated. The test images are 72 bits, not full bitstreams, and the configuration-time model of (1) has not been validated against hardware. No synthesis, resource-utilization, maximum-frequency, or board measurements are included. Coverage figures are for the directed test set and describe which RTL and scenarios were exercised, not

the absence of defects; functional coverage was defined by the authors and shares their blind spots. Missing coverpoints include the AXI write-data and response channels, response errors, ID mismatch, and watchdog expiry.

VII. Conclusion

We presented an RTL platform that sequences multiple full FPGA images from QSPI flash and advances between them on target-completion events, with a bounded abort path for control, identification, and task faults. A directed three-image testbench, fault-injection tests, and a scheduler covergroup give 79.7% RTL statement coverage on the integrated run and expose specific remaining gaps. The next steps are closing those gaps, validating the design against a real target device, and adding synthesis and timing results for a named FPGA part.

References

[1] AMD, *7 Series FPGAs Configuration User Guide*, UG470, v1.17, Dec. 2023.

[2] Arm, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, and *AMBA 4 AXI4-Stream Protocol Specification*, ARM IHI 0051, Arm Ltd.

[3] IEEE, *IEEE Standard for Test Access Port and Boundary-Scan Architecture*, IEEE Std 1149.1-2013, May 2013.

[4] G. Rodriguez-Canal, N. Brown, Y. Torres, and A. Gonzalez-Escribano, "Task-based preemptive scheduling on FPGAs leveraging partial reconfiguration," arXiv:2301.07615, 2023.

[5] B. Deng *et al.*, "JTAG-based remote configuration of FPGAs over optical fibers," arXiv:2401.15746, 2024.

[6] Project RTL and testbench sources (`programmer_pkg.sv`, `interrupt_controller.sv`, `time_multiplex_scheduler.sv`, `axi_lite_write_master.sv`, `fpga_multiphase_reconfig_top.sv`, `tb_multi_image_schedule.sv`), project repository.

[7] Xilinx, *MultiBoot with 7 Series FPGAs and SPI*, Application Note XAPP1247, v1.1, Feb. 2017.

[8] G. Charitopoulos, I. Koidis, K. Papadimitriou, and D. Pnevmatikatos, *Run-time management of systems with partially reconfigurable FPGAs*, *Integration*, vol. 57, pp. 34–44, Mar. 2017, doi: 10.1016/j.vlsi.2016.11.008.

[9] T. Li, F. Zhang, W. Guo, M. Sun, and L. Chen, *A Survey: FPGA-Based Dynamic Scheduling of Hardware Tasks*, *Chinese Journal of Electronics*, vol. 30, no. 6, pp. 991–1007, Nov. 2021, doi: 10.1049/cje.2021.07.021.

[10] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2023, Feb. 2024, doi: 10.1109/IEEESTD.2024.10458102.

[11] A.-C. Cheng, C.-C. Yen, C. G. Val, S. Bayless, A. J. Hu, I. H.-R. Jiang, and J.-Y. Jou, *Efficient coverage-driven stimulus generation using simultaneous SAT solving, with application to SystemVerilog*, *ACM Transactions on Design Automation of Electronic Systems*, vol. 20, no. 1, Art. 7, pp. 1–23, Nov. 2014, doi: 10.1145/2651400.